



Institución
Universitaria
Reacreditada en Alta Calidad

Neural Architecture Search

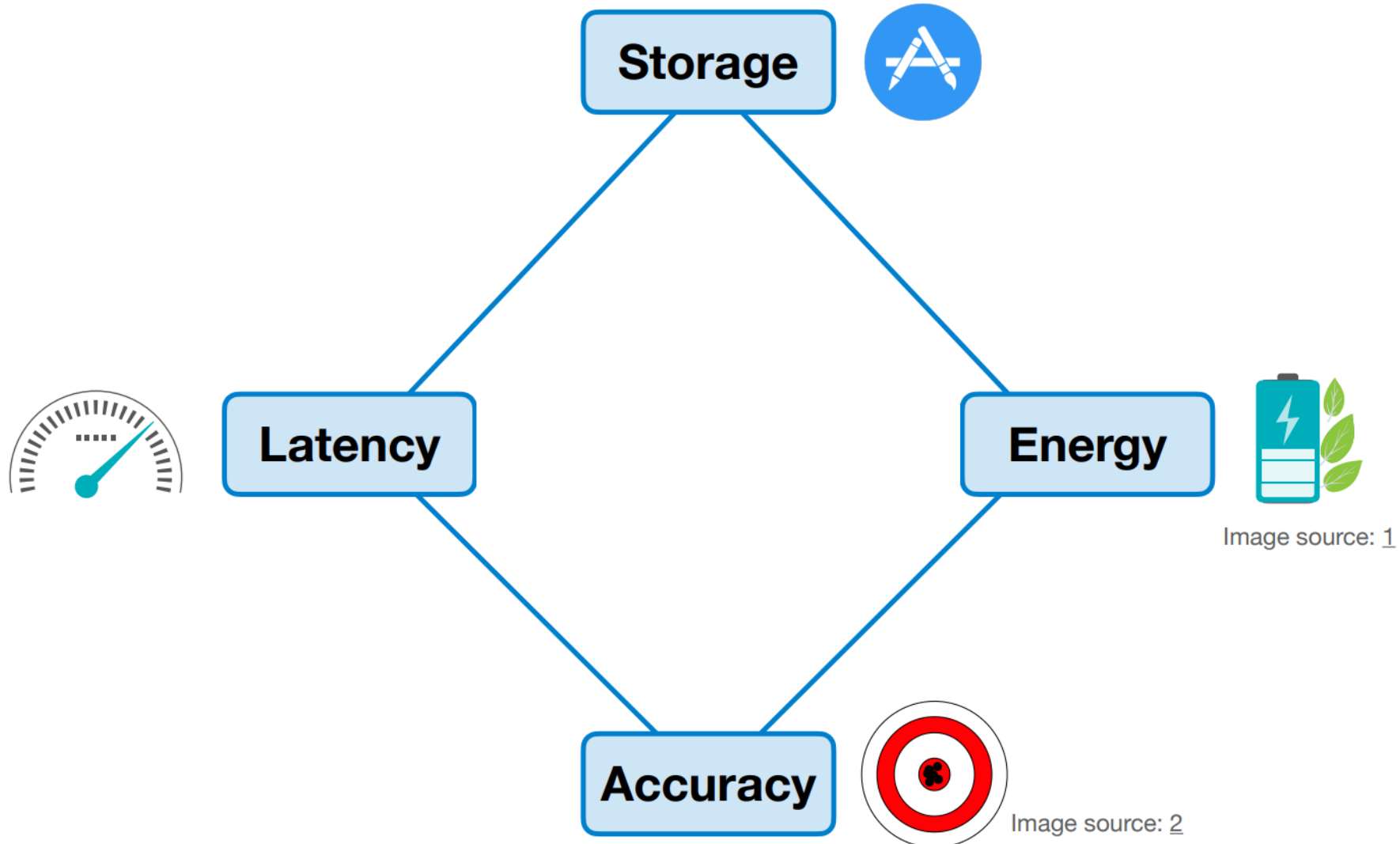
INTEGRACIÓN DE ML EN EMBEBIDOS Y EDGE COMPUTING

Somos Innovación Tecnológica con *Sentido Humano*

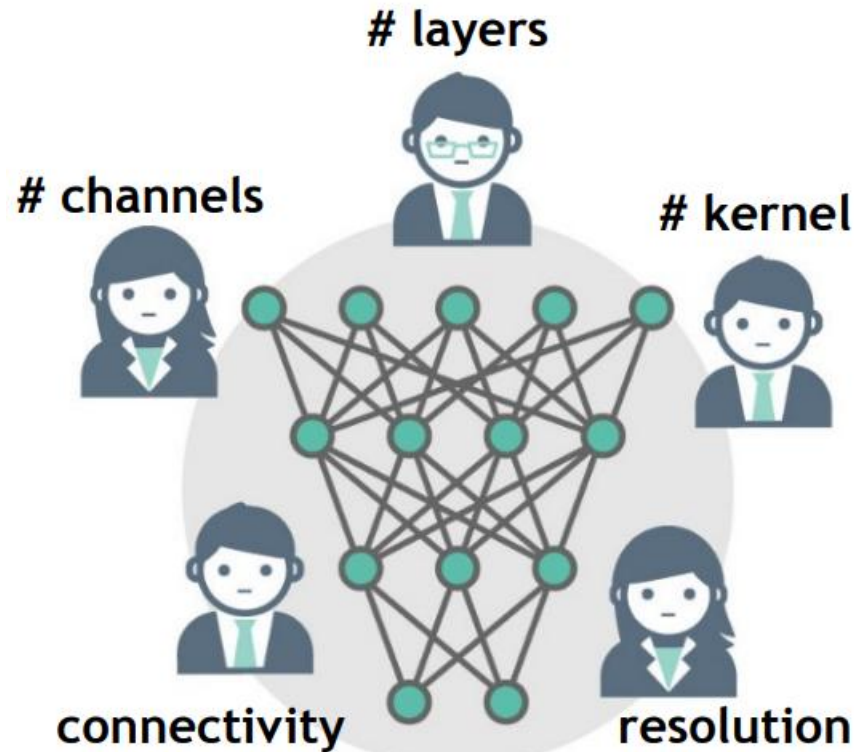


Alcaldía de Medellín

NAS – Neural Architecture Search



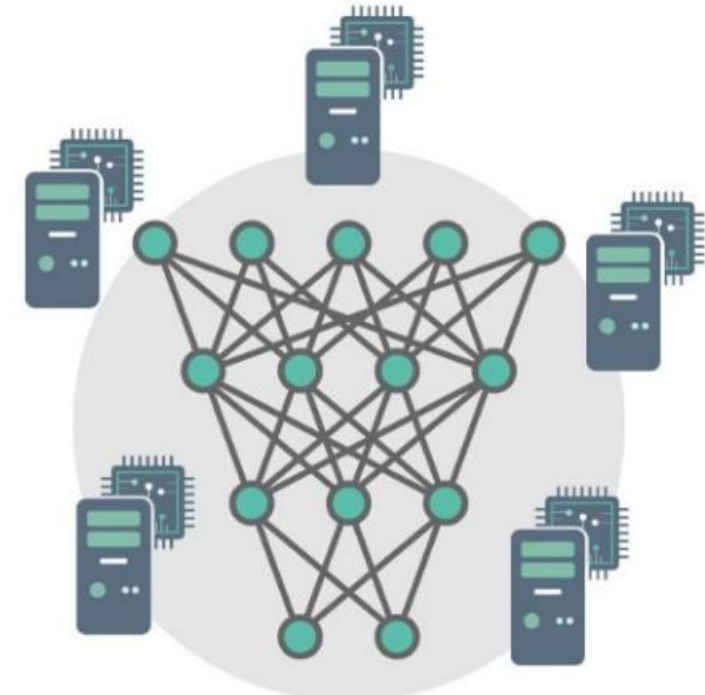
NAS – Neural Architecture Search



Use Human Expertise

Manual Architecture Search

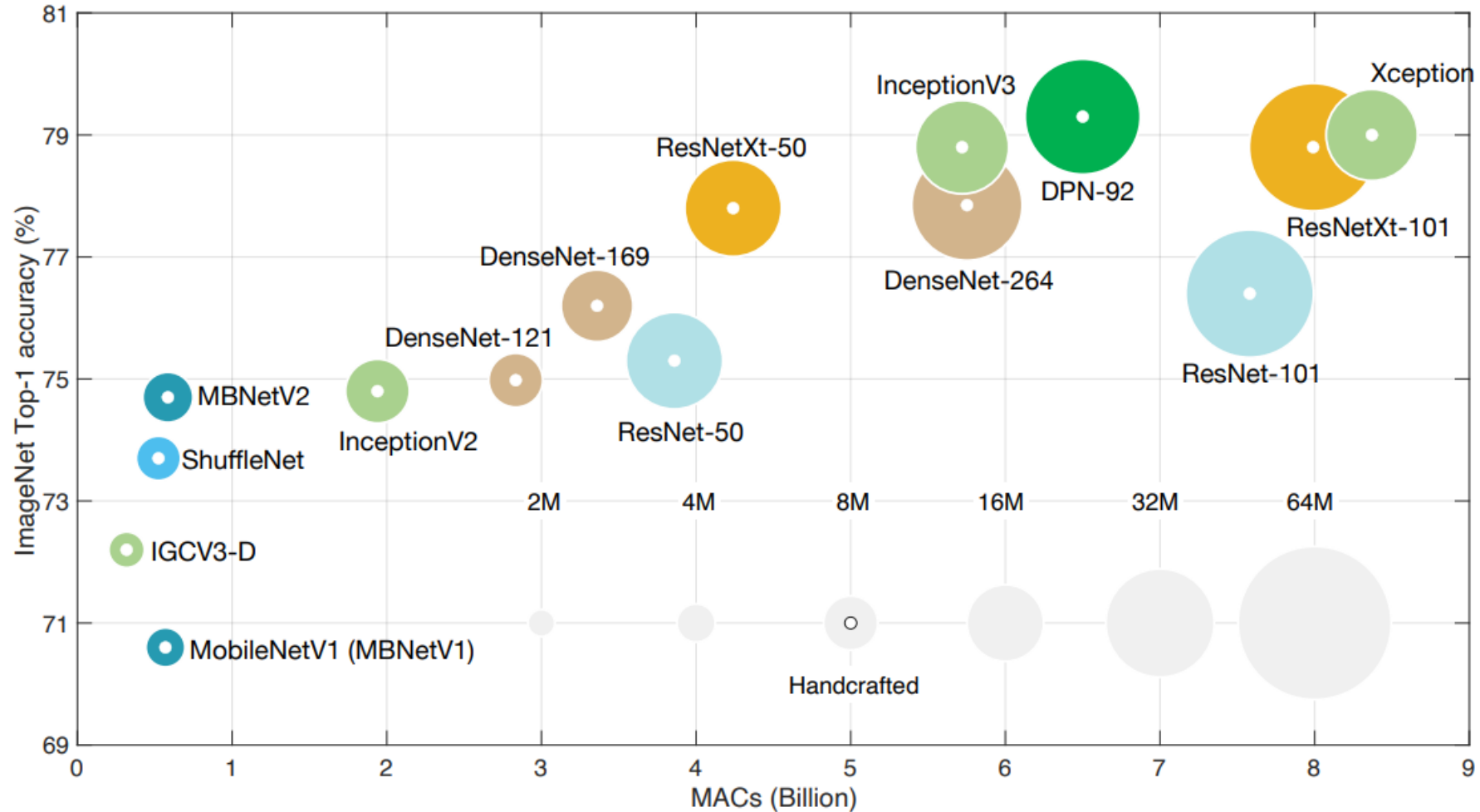
Huge design space, manual design is unscalable



Use Machine Learning

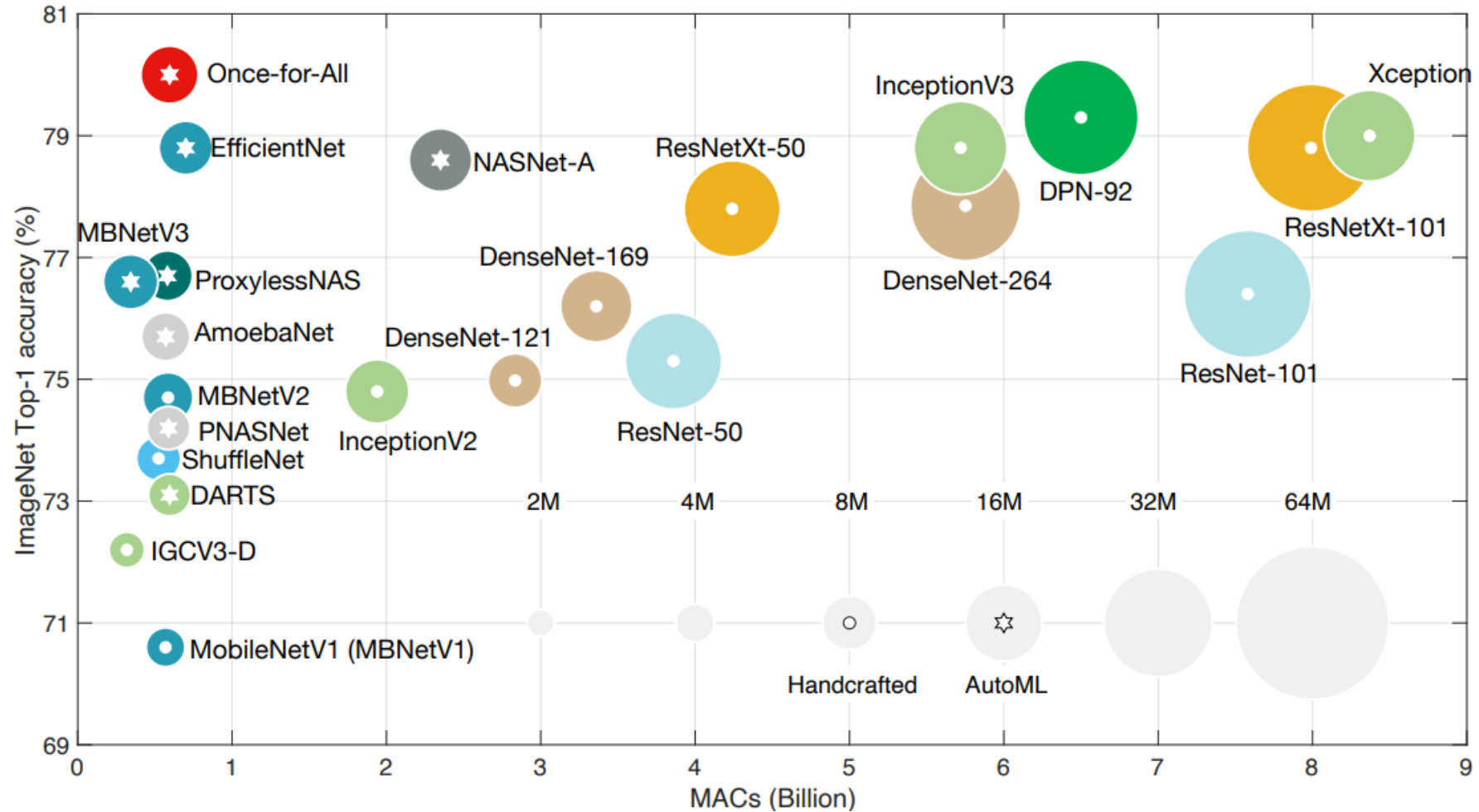
Automatic Architecture Search

Manually-designed NNs



Accuracy-efficiency trade-off on ImageNet

From Manual Design to Automatic Design



Huge design space, manual design is unscalable



Contenido

1. Neural Architecture Search (NAS)
2. Métodos de Búsqueda
3. Búsqueda de arquitectura
4. Hardware-aware NAS
5. Otras técnicas

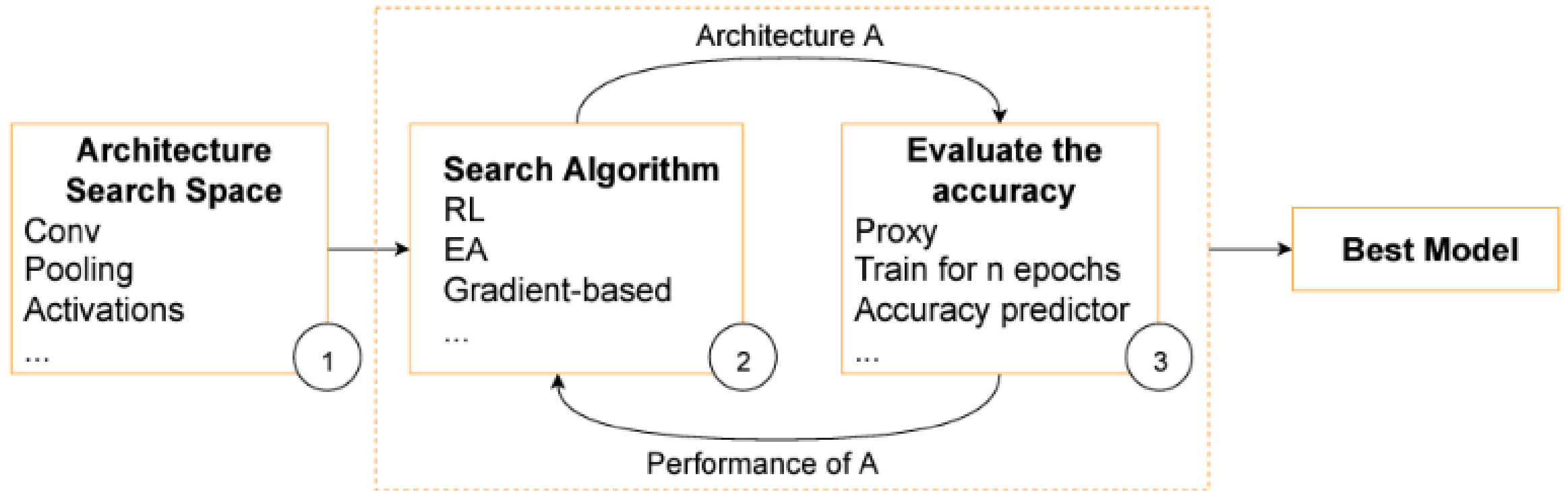
NAS – Neural Architecture Search

Un método NAS consiste de:

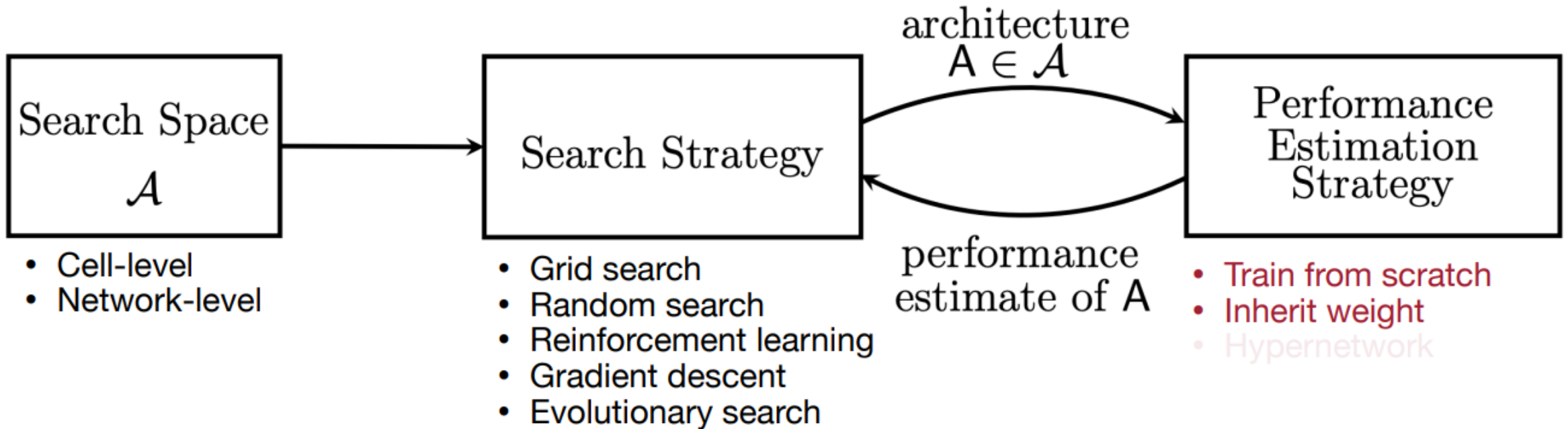
1. Espacio de búsqueda (Search Space): estructura y conjunto de operaciones primitivas construidas manualmente. Puede ser aplicado a celdas o a capas. Se definen los tipos de capas o redes que pueden ser diseñadas y optimizadas.
2. Estrategia de búsqueda (Search Strategy): algoritmo de búsqueda a través del cual se busca una red en el espacio de búsqueda.
3. Fase de evaluación: donde la red encontrada es evaluada para su desempeño.

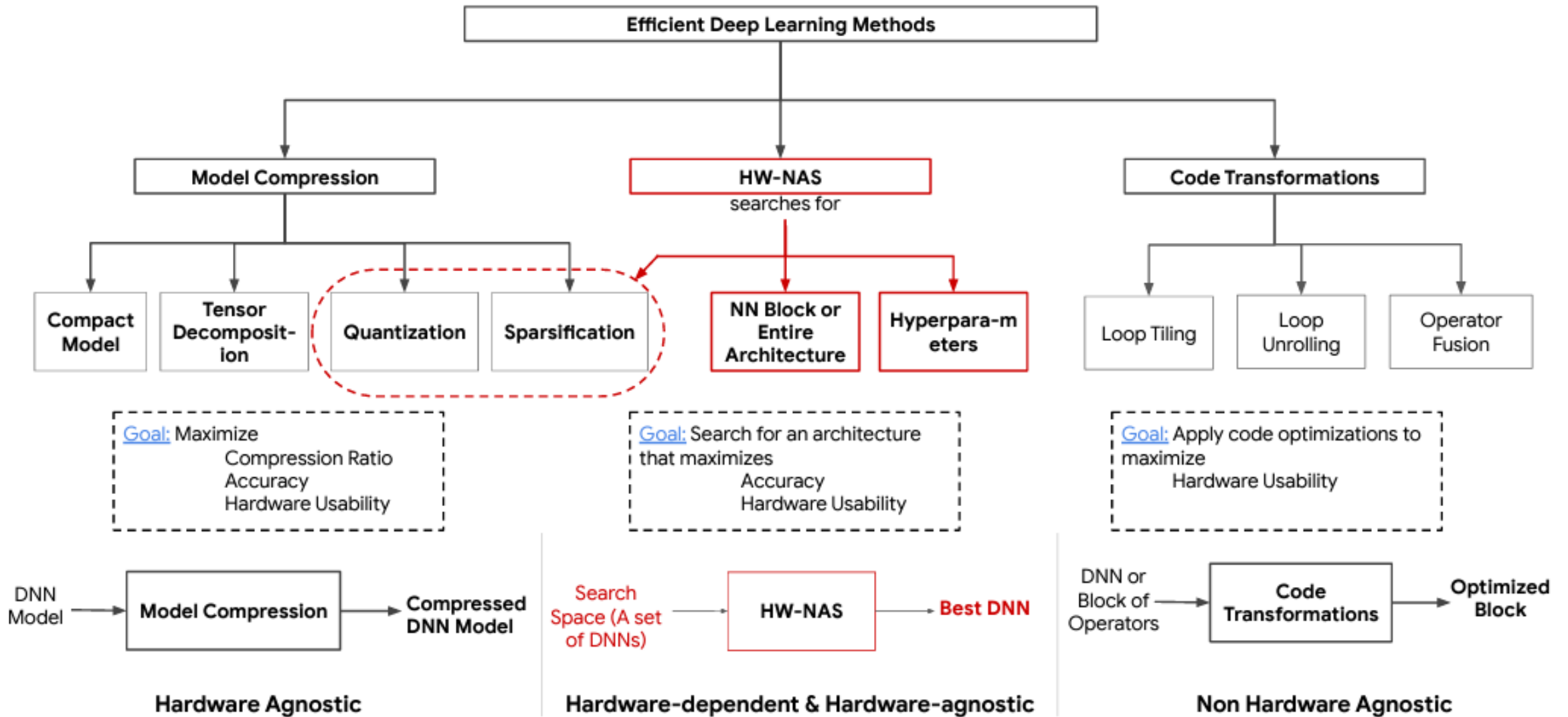
Por lo general las técnicas NAS están enfocadas en encontrar la red neuronal que entregue mejor porcentaje de acierto o desempeño. Y estas soluciones llevan a redes neuronales gigantes. Se deben buscar técnicas NAS para restricciones en hardware.

NAS - Concepto



NAS - Concepto







Contenido

1. Neural Architecture Search (NAS)
2. **Métodos de Búsqueda**
3. Búsqueda de arquitectura
4. Hardware-aware NAS
5. Otras técnicas

Métodos de búsqueda

Operation Set /
Candidate Operations:

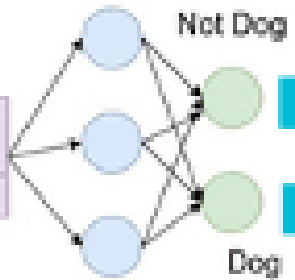
op1

op2

op3



Image cc-by: Vonuzanka

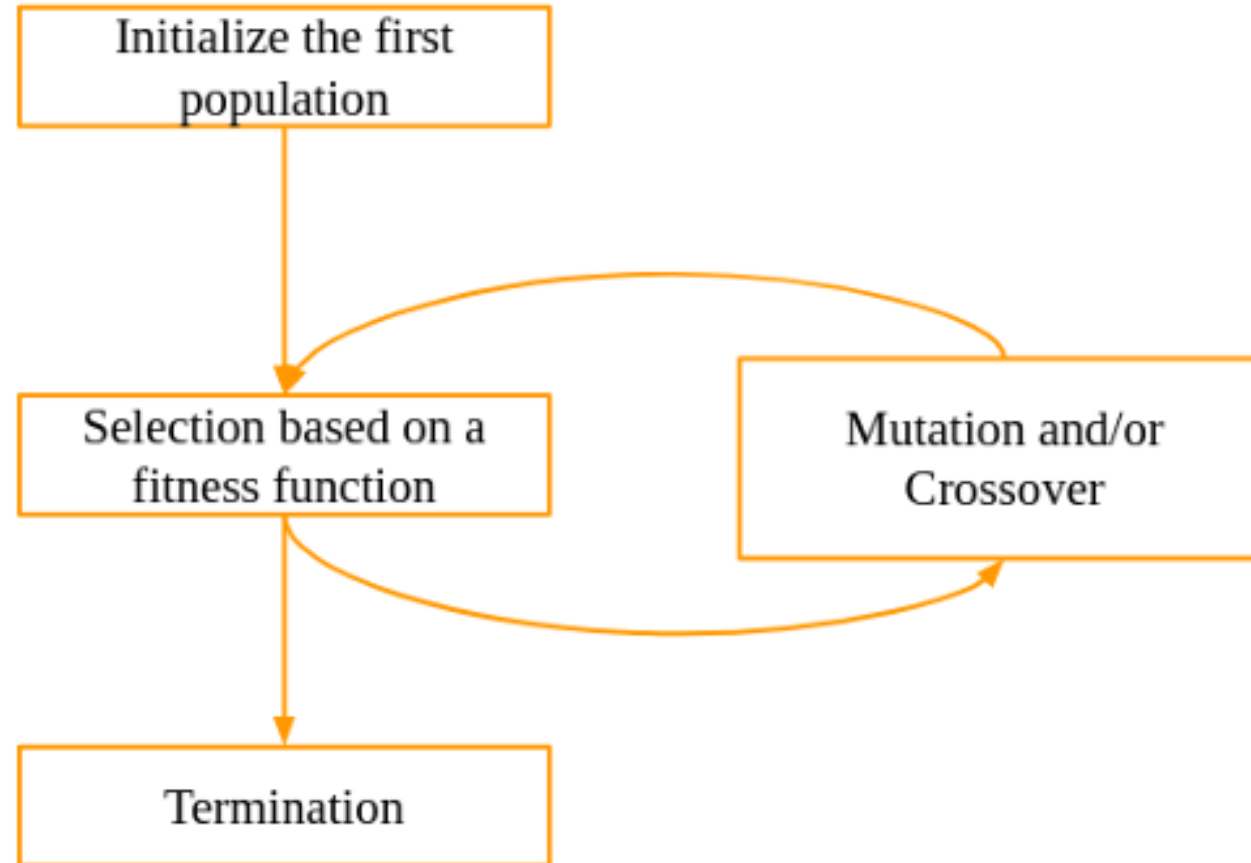


ANIMATE

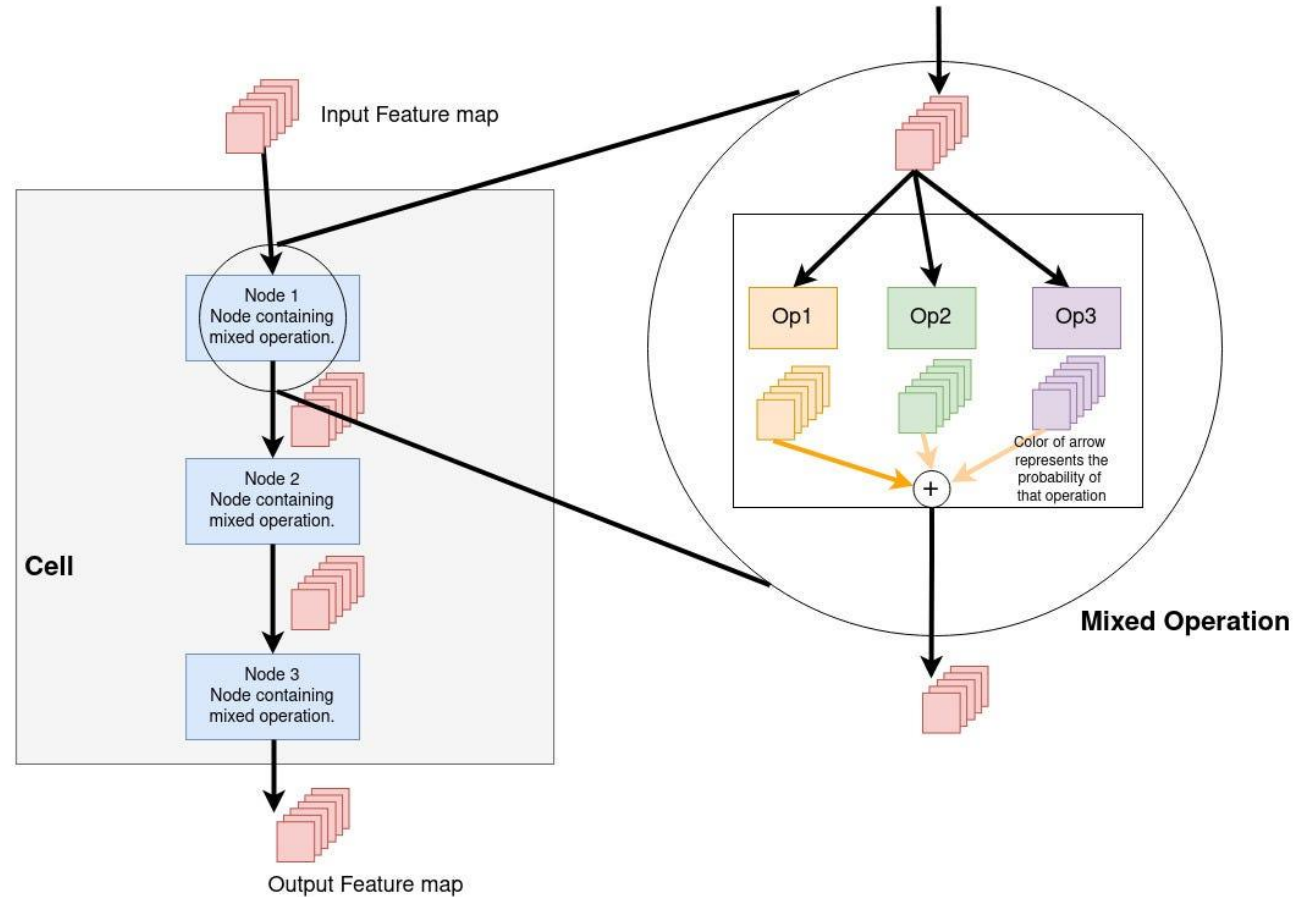
Métodos de búsqueda - RL



Métodos de búsqueda - Optimización Heurística



DARTS: Differentiable Neural Architecture Search



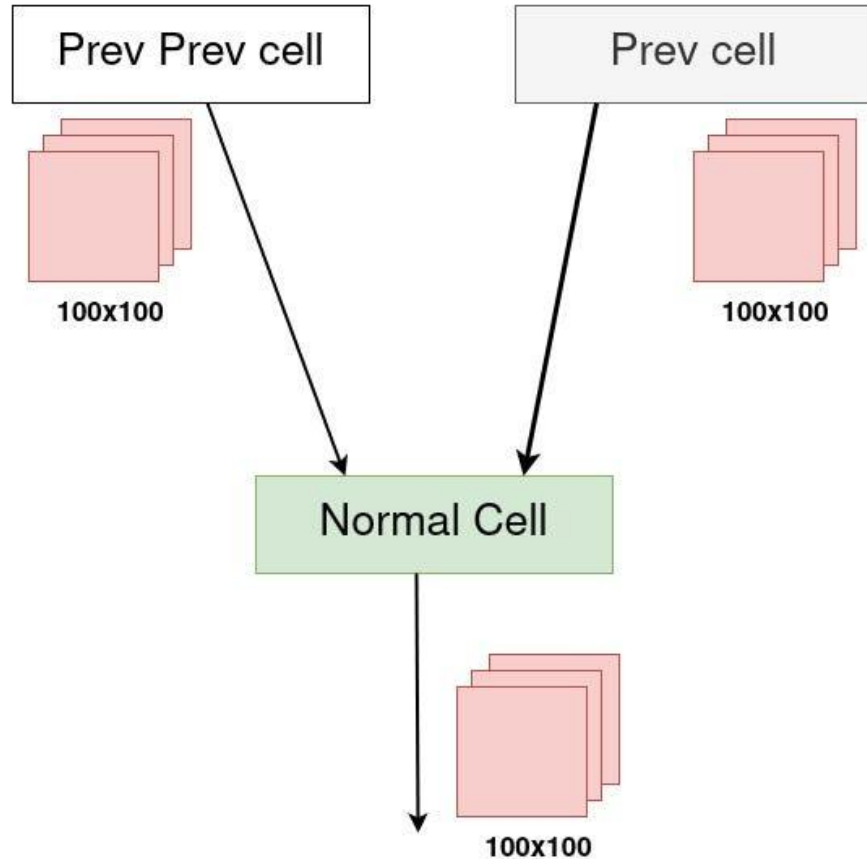
Searching Cell Architectures

<https://towardsdatascience.com/intuitive-explanation-of-differentiable-architecture-search-darts-692bda1cc69c/>

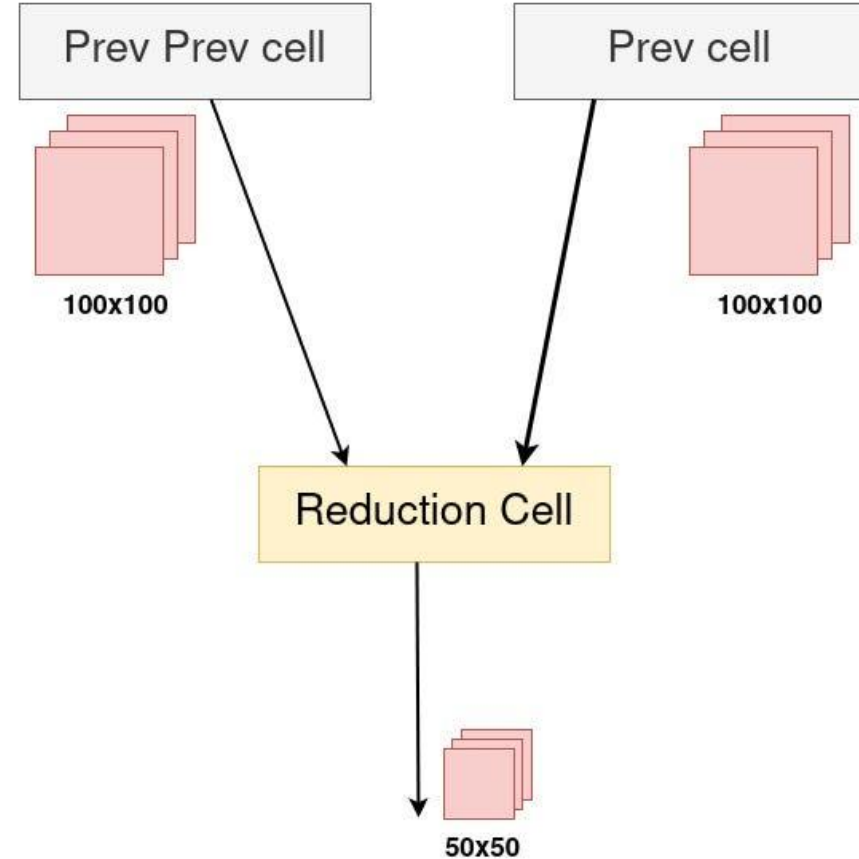
Series de Innovación Tecnológica con

Sentido Humano

DARTS: Differentiable Neural Architecture Search

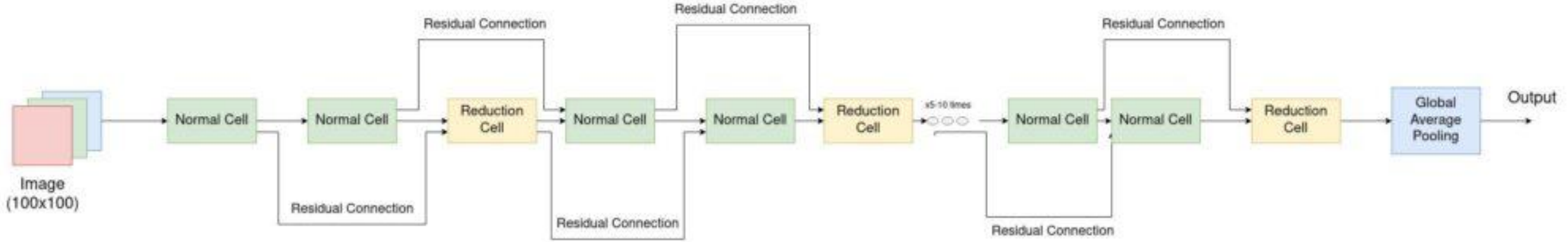


Normal cell: Bloque que calcula el mapa de características de una imagen.



Reduction cell: Bloque normal con las dimensiones del mapa de características reducida.

DARTS: Differentiable Neural Architecture Search



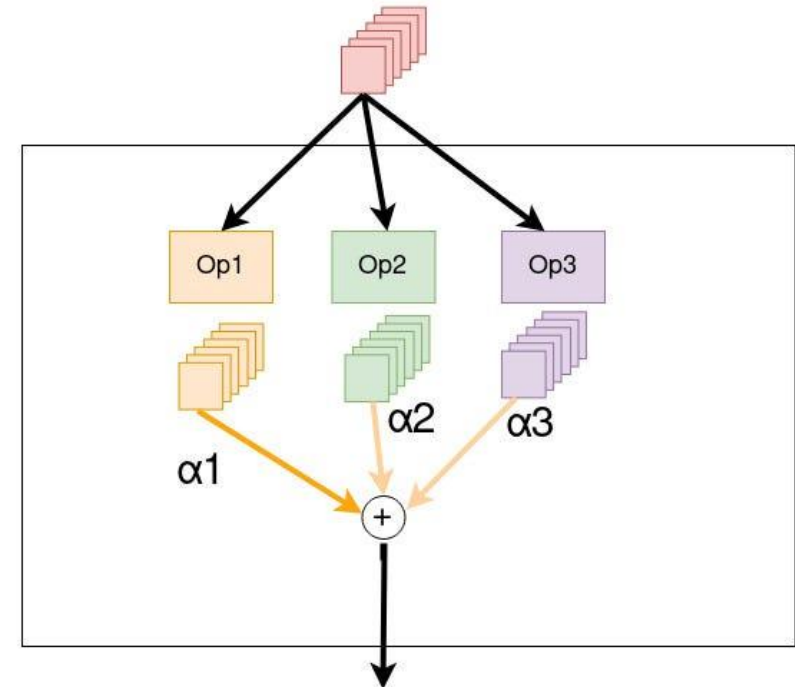
DARTS: Differentiable Neural Architecture Search

El resultado de la operación se denomina «operación mixta» y se calcula multiplicando el resultado de estas operaciones por sus respectivas probabilidades

$$\bar{o}^{(i,j)}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} o(x)$$

"To make the search space continuous, we relax the categorical choice of a particular operation to a softmax over all possible operations:" – DARTS paper

$\mathcal{O} = \{\text{conv_3x3, max_pool_3x3, dilated_conv_5x5}\}$

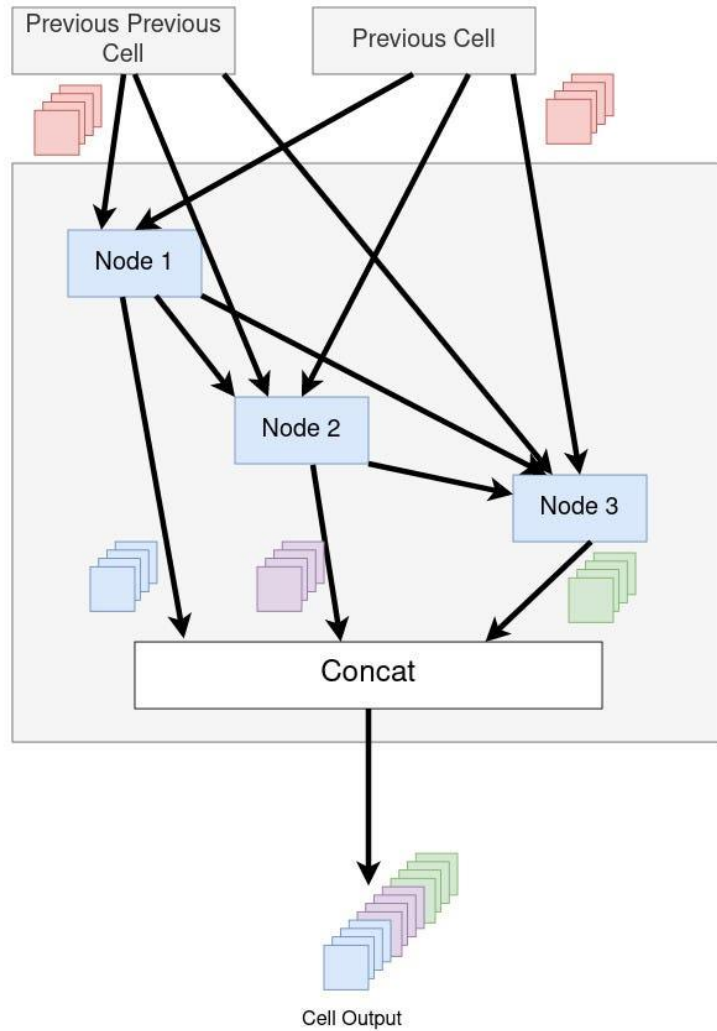


$$\alpha_1^* \text{ (orange) } + \alpha_2^* \text{ (green) } + \alpha_3^* \text{ (purple) } = \text{ (red) }$$

"Each intermediate node is computed based on all of its predecessors." – DARTS paper

Somos Innovación Tecnológica con *Sentido Humano*

DARTS: Differentiable Neural Architecture Search



$$o^{(i,j)} = \operatorname{argmax}_{o \in \mathcal{O}} \alpha_o^{(i,j)}$$

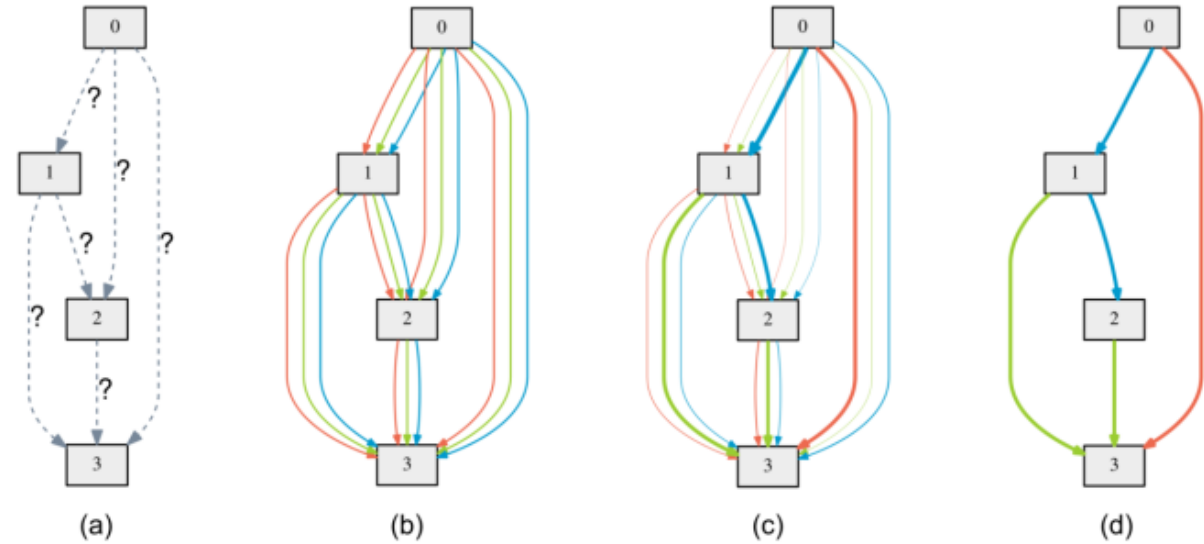


Figure 1: An overview of DARTS: (a) Operations on the edges are initially unknown. (b) Continuous relaxation of the search space by placing a mixture of candidate operations on each edge. (c) Joint optimization of the mixing probabilities and the network weights by solving a bilevel optimization problem. (d) Inducing the final architecture from the learned mixing probabilities.

Note how each node has output from all the previous nodes as its input.

DARTS: Differentiable Neural Architecture Search

Bilevel Optimization

"After relaxation, our goal is to jointly learn the architecture α and the weights w within all the mixed operations (e.g. weights of the convolution filters)." – DARTS Paper

$$\begin{aligned} \min_{\alpha} \quad & \mathcal{L}_{val}(w^*(\alpha), \alpha) \\ \text{s.t.} \quad & w^*(\alpha) = \operatorname{argmin}_w \mathcal{L}_{train}(w, \alpha) \end{aligned}$$

Note how the optimization of alphas and layer weights are done on training and validation set.

DARTS: Differentiable Neural Architecture Search

Bilevel Optimization

"After relaxation, our goal is to jointly learn the architecture α and the weights w within all the mixed operations (e.g. weights of the convolution filters)." – DARTS Paper

$$\begin{aligned} \min_{\alpha} \quad & \mathcal{L}_{val}(w^*(\alpha), \alpha) \\ \text{s.t.} \quad & w^*(\alpha) = \operatorname{argmin}_w \mathcal{L}_{train}(w, \alpha) \end{aligned}$$

Note how the optimization of alphas and layer weights are done on training and validation set.

DARTS: Differentiable Neural Architecture Search

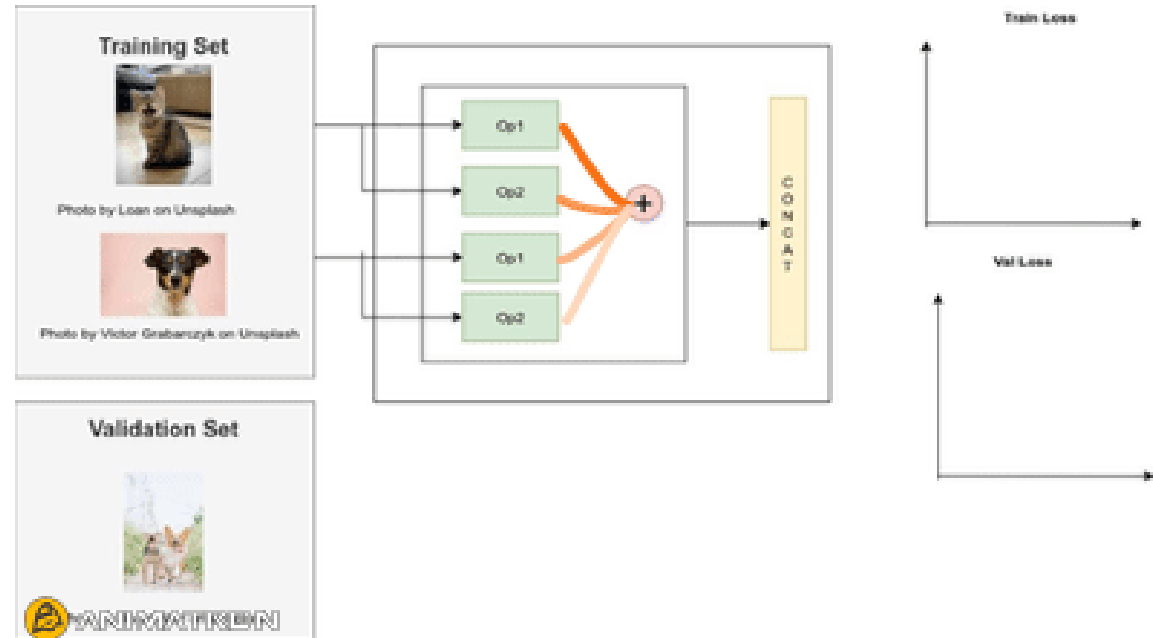
Bilevel Optimization

“Evaluating the architecture gradient exactly can be prohibitive due to the expensive inner optimization. We, therefore, propose a simple approximation scheme as follows:” DARTS Paper

$$\nabla_{\alpha} L_{val}(w^*(\alpha), \alpha)$$

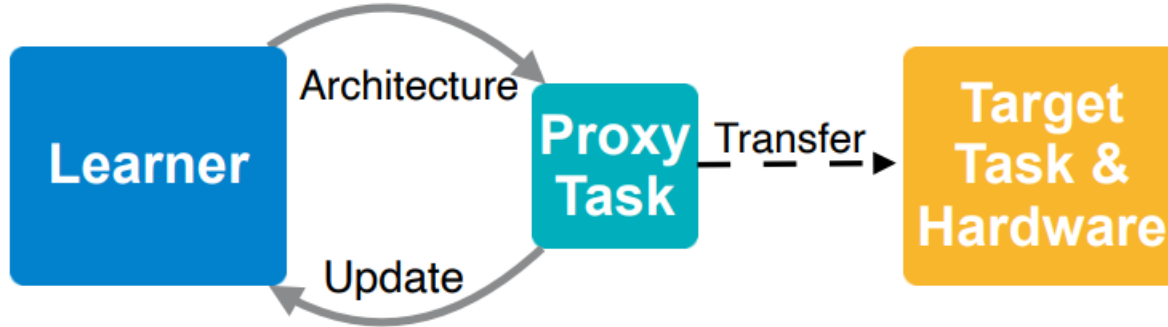
α : parameter for operation weight

$w^*(\alpha)$: optimal conv weights for specific α



Efficiently Search on Target Task & Hardware

Proxy task: sub-optimal



Los métodos anteriores de NAS son **muy costosos**.

- NASNet: 48,000 GPU horas, aun sobre Cifar $\approx$ 5 years on single GPU
- DARTS: memoria de 100GB GPU si se busca directametine sobre ImageNet.

Entonces, los trabajos previos debían utilizar “proxy tasks”:

- CIFAR-10
- Small architecture space (p.e. baja profundidad).
- Pocas epocas de entrenamiento.
- Flops y cantidad de parámetros.



Contenido

1. Neural Architecture Search (NAS)
2. Métodos de Búsqueda
3. **Búsqueda de arquitectura**
4. Hardware-aware NAS
5. Otras técnicas

Architectural Search Space

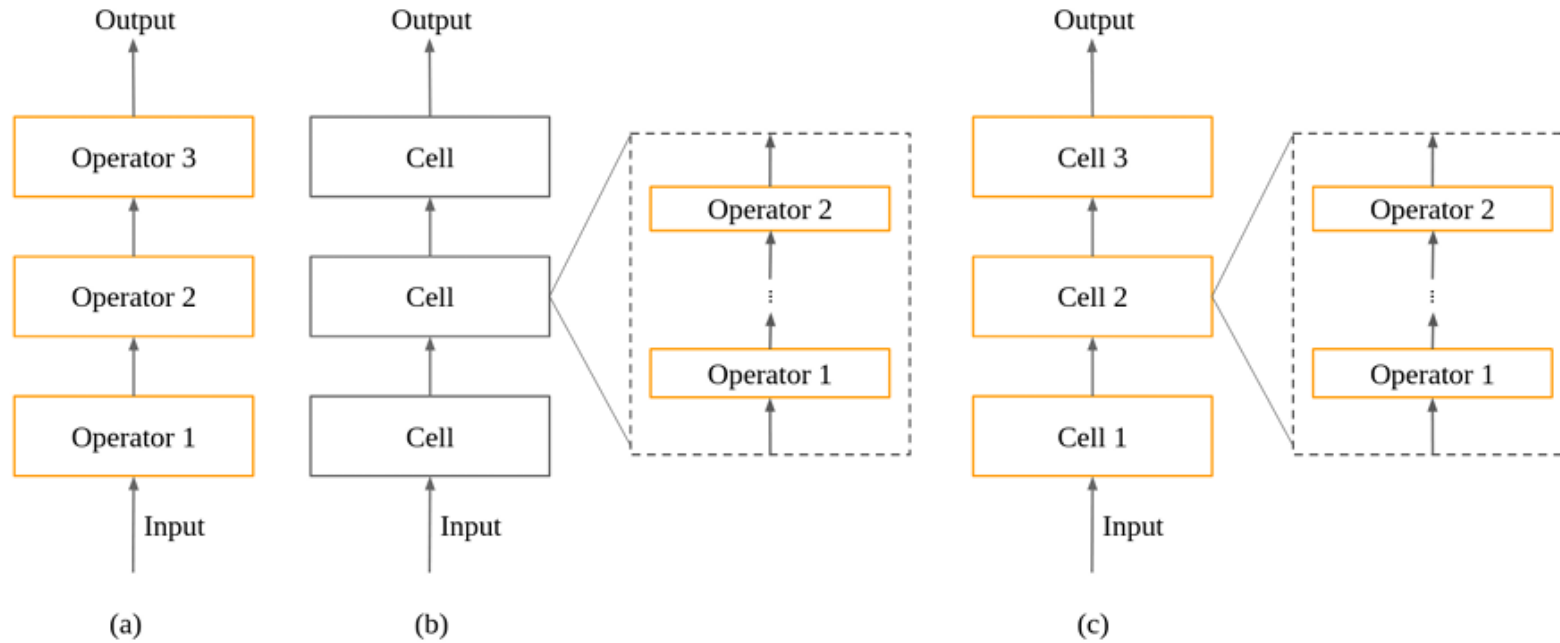


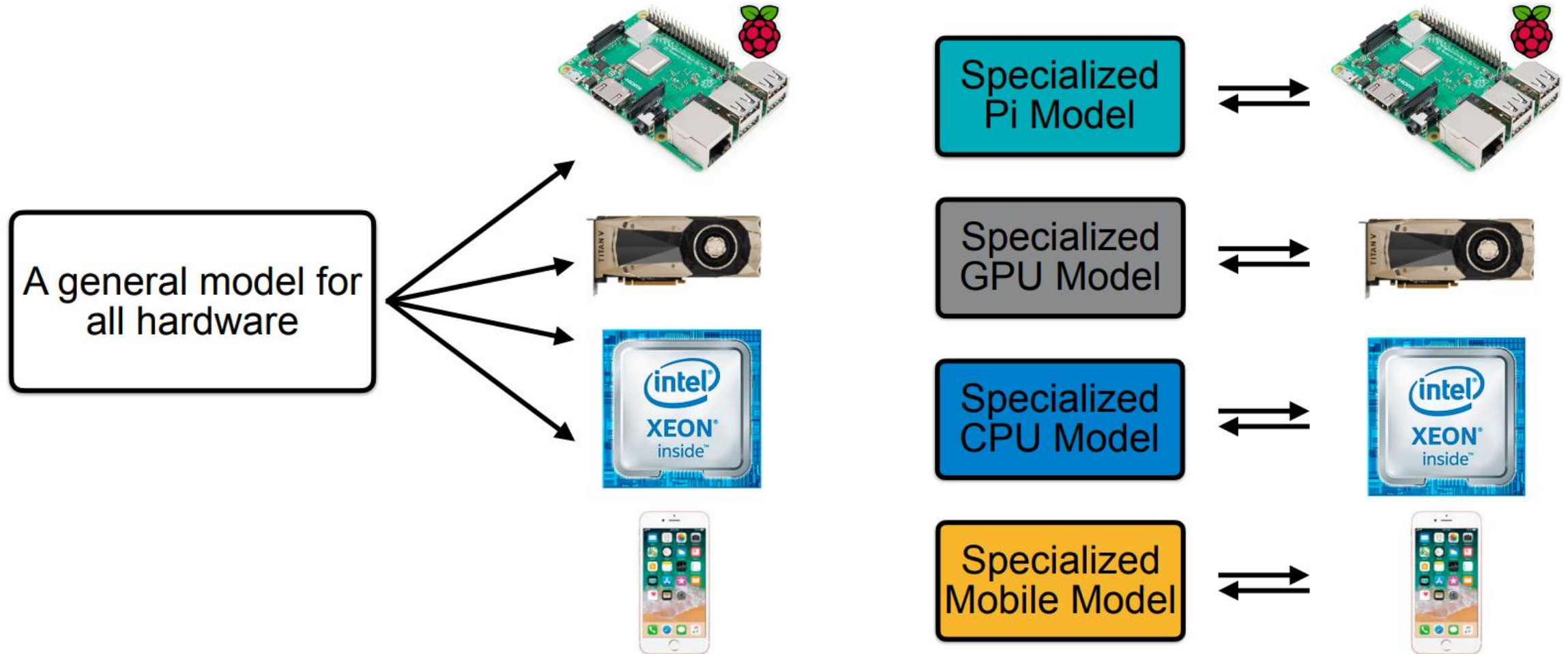
Fig. 10. Architecture search spaces types. (a) Global search space, (b) Cell-based search space, and (c) Hierarchical search space. In orange the operators considered during the search.



Contenido

1. Neural Architecture Search (NAS)
2. Métodos de Búsqueda
3. Búsqueda de arquitectura
4. **Hardware-aware NAS**
5. Otras técnicas

De modelo generales a especializados



General purpose: inefficient

Specialized: efficient

NAS – Constraint Hardware

Un método NAS para restricción de hardware es:

El proceso de optimización del diseño de la arquitectura red neuronal se debe enfocar en: espacio de memoria disponible, numero de FLOPs, numero de MACs (Multiply-ACumulate operations) o latencia de la inferencia.

Hardware-aware NAS

Problemas:

1. La cantidad de tipos variados de datos y tareas que requieren diferentes diseños y optimizaciones de arquitecturas neuronales.
2. La gran cantidad de plataformas de hardware que hacen difícil el diseño de una arquitectura globalmente eficiente.

Processor	Usecase	Compute	Memory	Power	Cost
Nvidia 1080Ti GPU [3]	Desktop	10 TFLOPs/Sec	11 GB	250 W	\$700
Intel i9-9900K CPU [6, 5]	Desktop	500 GFLOPs/Sec	256 GB	95 W	\$499
Google Pixel 1 (Arm CPU) [10]	Mobile	50 GOPs/Sec	4 GB	~ 5 W	–
Raspberry Pi (Arm CPU) [11]	Hobbyist	50 GOPs/Sec	1 GB	1.5 W	–
Micro:Bit (Arm MCU) [8]	IoT	16 MOPs/Sec	16 KB	~ 1 mW	\$1.75
Arduino Uno (Microchip MCU) [1]	IoT	4 MOPs/Sec	2 KB	~ 1 mW	\$1.14

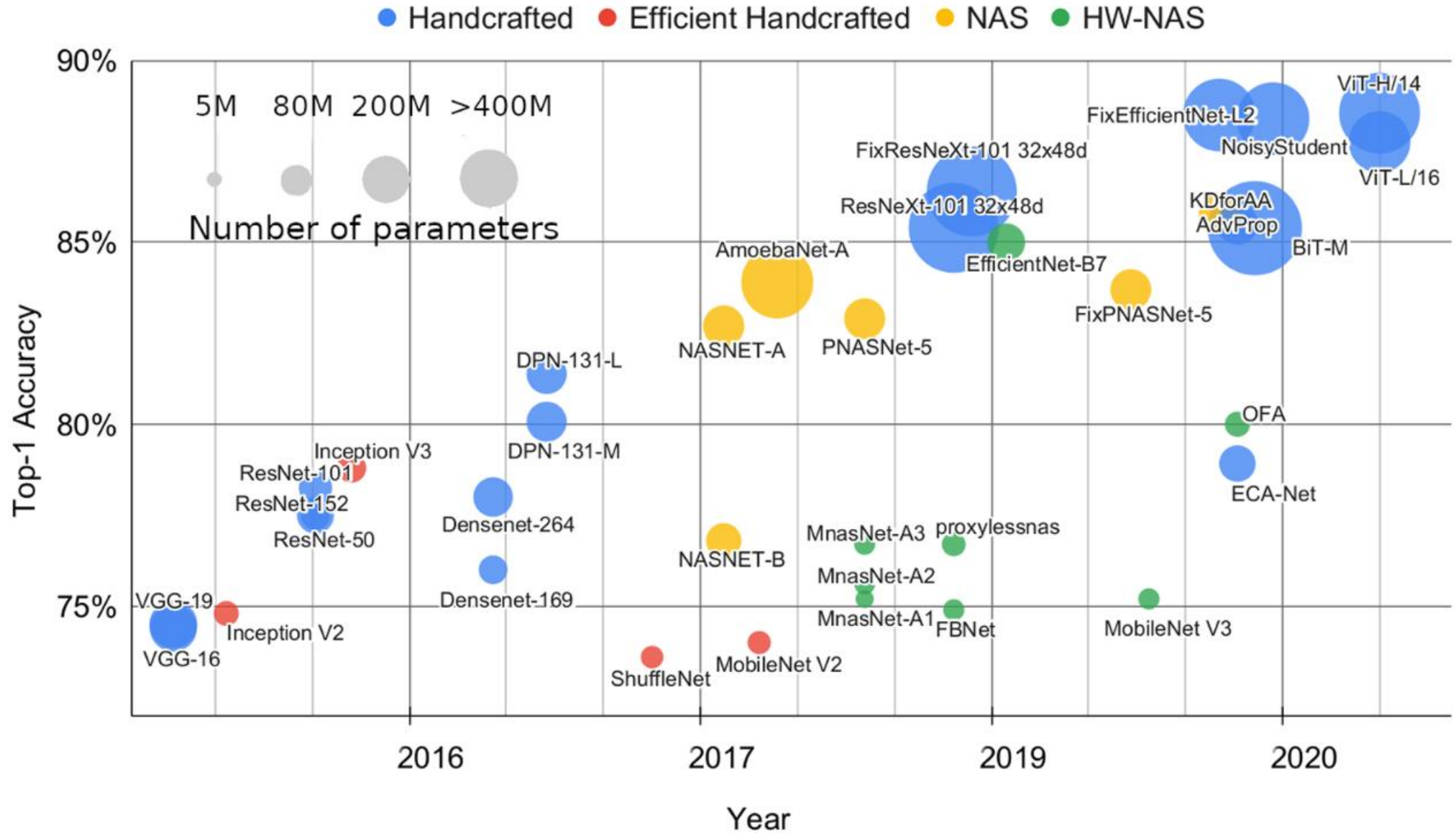
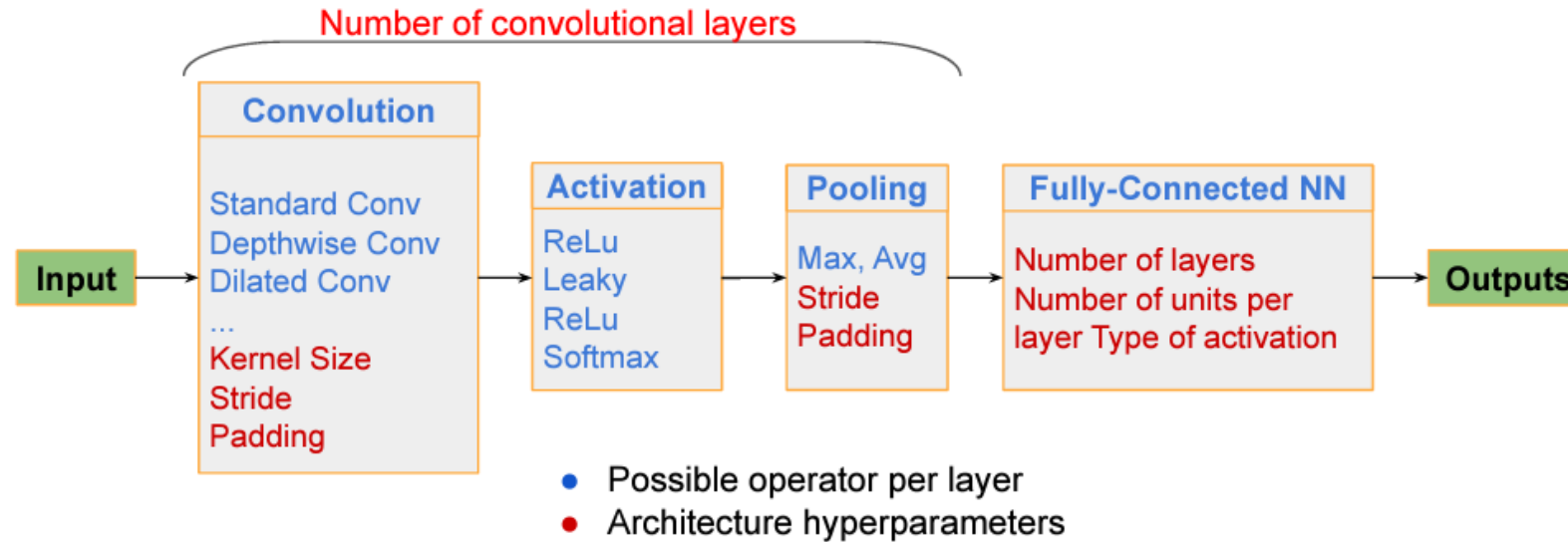


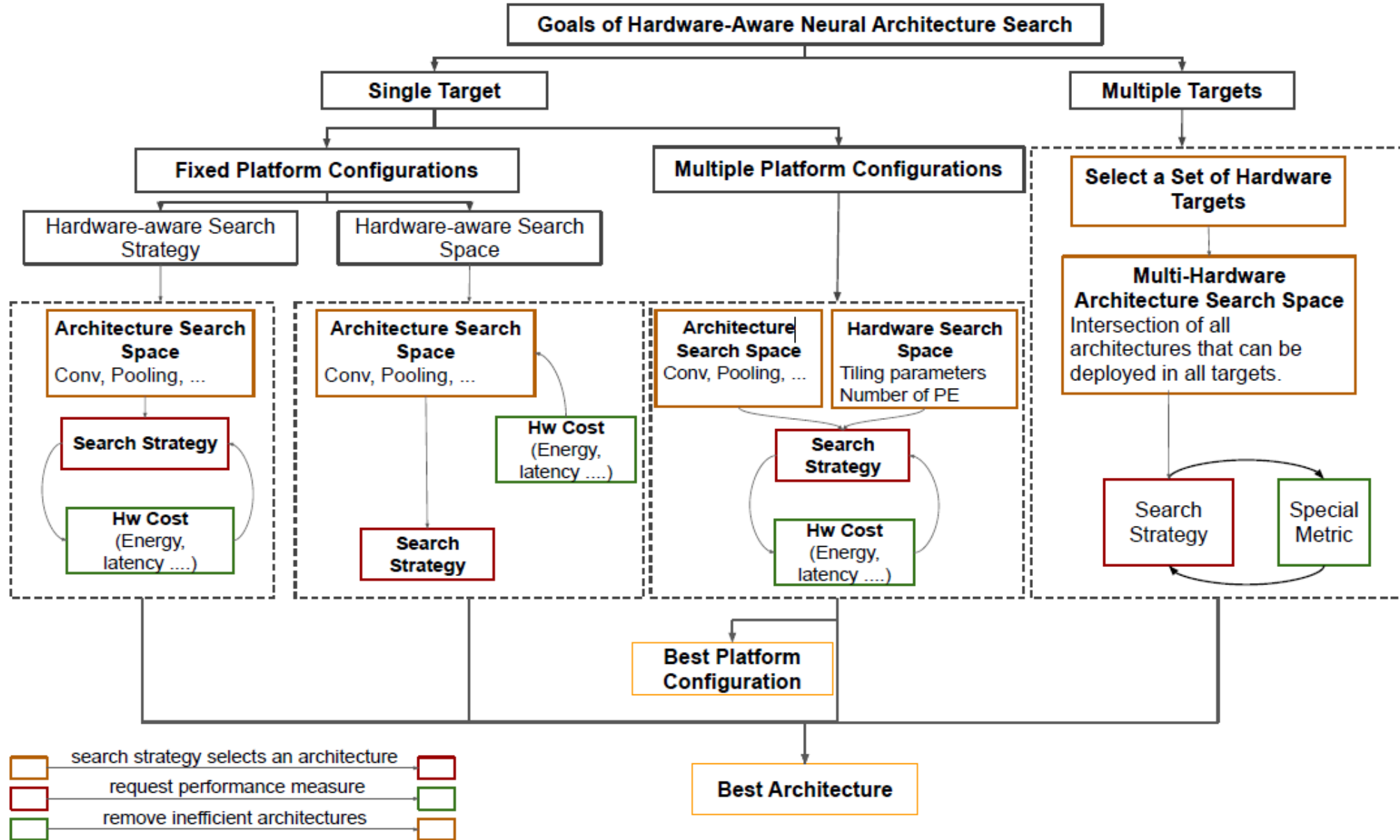
Tabla descriptiva de HW-NAS

Background	Efficient Deep Learning	Model Compression HW-NAS Code Transformations
	Algorithms	Reinforcement Learning Evolutionary Algorithm
Taxonomy of HW-NAS	Classification of HW-NAS based on their Goals	
Search Spaces	Architecture Search Space	
	Hardware Search Space	
HW-NAS Problem Formulation	Single-Objective Optimization	Two-Stage Search Constrained Optimization
	Multi-Objective Optimization	Scalarization NSGA-II
Search Strategies	Search Algorithm	Reinforcement Learning Evolutionary Algorithm Gradient-based Methods Bayesian Optimization & Random Search
		Over-parameterized Networks & their training
	Runtime Performance Optimization Strategies	Early Stopping Hot Start Proxy Datasets Accuracy Prediction Models
Hardware Cost Estimation Methods	Hardware Constraints Collection Techniques	Real-time measurements Lookup Table Analytical Estimation Prediction Models

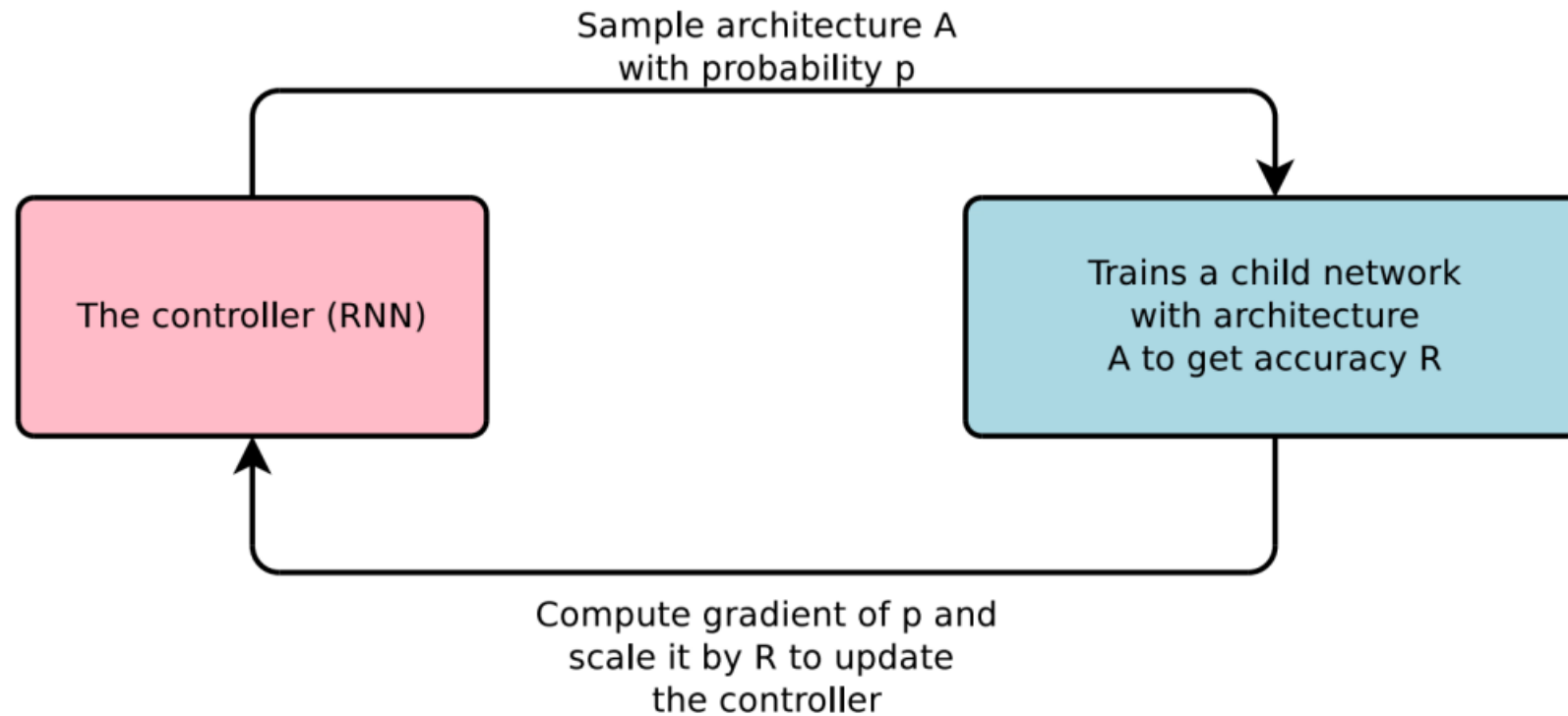
Optimizar CNN



CNN genérica. Para cada capa se selecciona un operador dentro de una lista predefinida (convolución, maxpooling, batch_normalization...)



Estrategia Estimación del acierto



CIFAR10

Train 12,800 model architectures

22,400 GPU-hours

Large-Scale Datasets

ImageNet

COCO

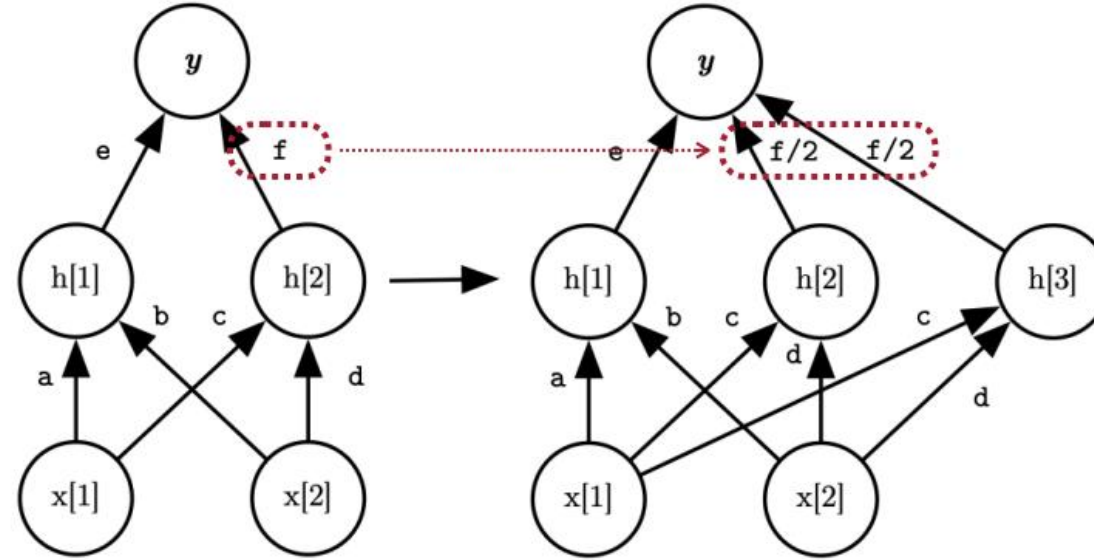
...



- Entrenar el modelo desde cero.
- Evaluar el modelo entrenado con los datos de test y obtener acierto R
- Drawback: coste prohibitivo de entrenamiento

Estrategia Estimación del acierto

Herencia de pesos



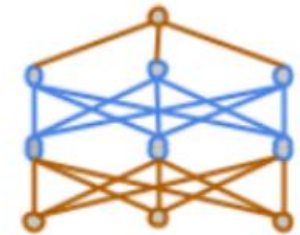
Net2Wider

Original Model

Layers that Initialized as
Identity Mapping

A Deeper Model Contains
Identity Mapping Initialized Layers

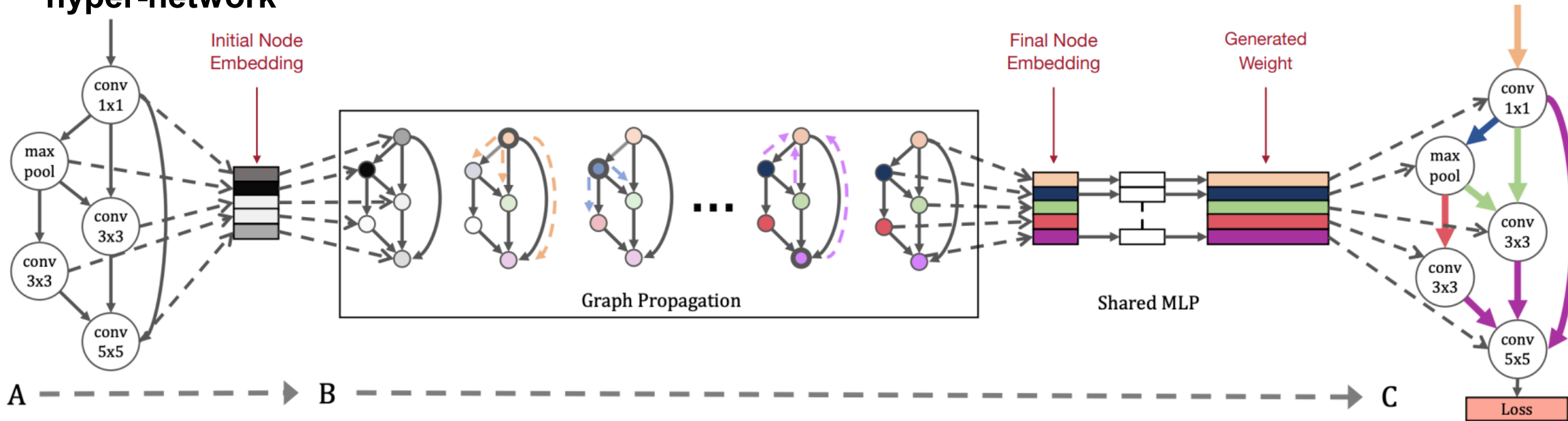
Net2Deeper



- Hereda pesos de un model padre para reducir el costo de entrenamiento en vez de entrenas desde cero.

Estrategia Estimación del acierto

hyper-network



- En cada paso de entrenamiento, una arquitectura aleatoria es mostrada del espacio de búsqueda (search space).
- La hyper-network genera pesos basados en la arquitectura del modelo.
- Usando gradiente descendente actualizar los pesos de la hyper-network.



Contenido

1. Neural Architecture Search (NAS)
2. Métodos de Búsqueda
3. Búsqueda de arquitectura
4. Hardware-aware NAS
5. Otras técnicas

E-DNAS

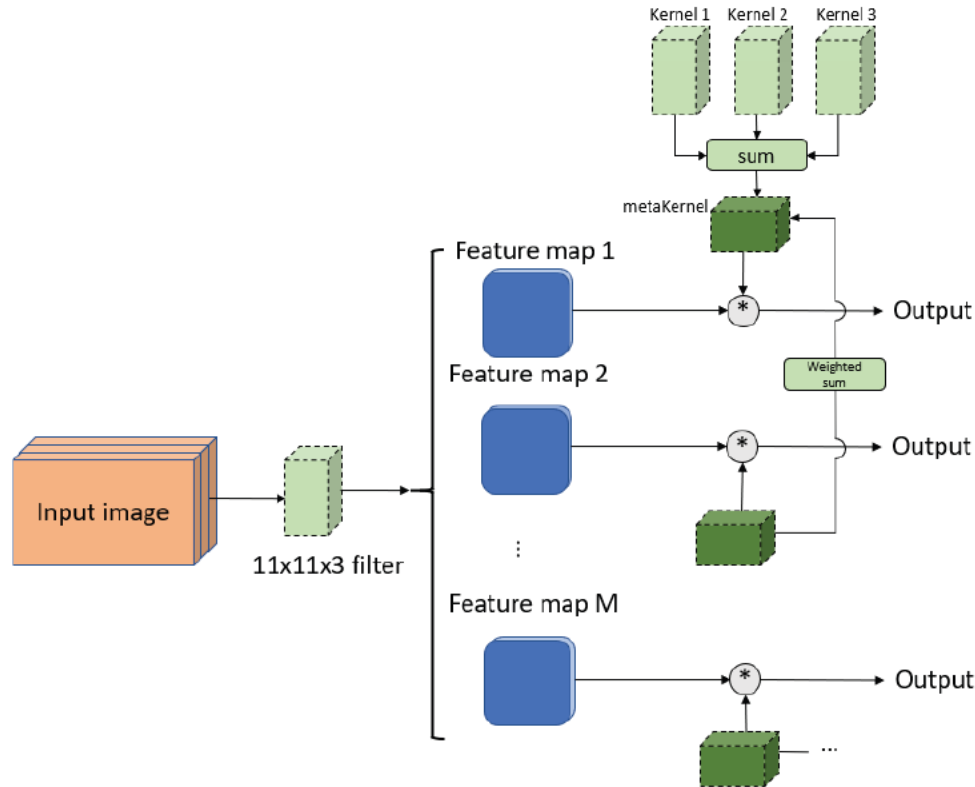


FIG. 1: **General overview of E-DNAS.** Our approach has two main building blocks: a depth-aware convolution with a high resolution 11×11 kernel followed by pairwise learning of meta-kernels with loopy flow of information on each iteration between training paths.

Algorithm 1: The search architecture methodology

Result: Find weights w_i and architecture probability parameters α to optimize the global loss function (13), given a defined search space with a combination of operations $\bar{o}^{(i,j)}$, defined in Eq. (9), a latency budget and an input dataset.

random initialization of α parameters

while *not converge* **do**

 Similar to [10], we generate the kernel candidates.

 Calculate Loss through Eq. (13).

 Calculate $\partial L / \partial w_a$ and $\partial L / \partial \alpha$.

 Update weights and architecture probability parameters α .

 Update Kernels using Eq. 3 and Eq. 5.

end

Extract more optimal architecture from learned α parameters.

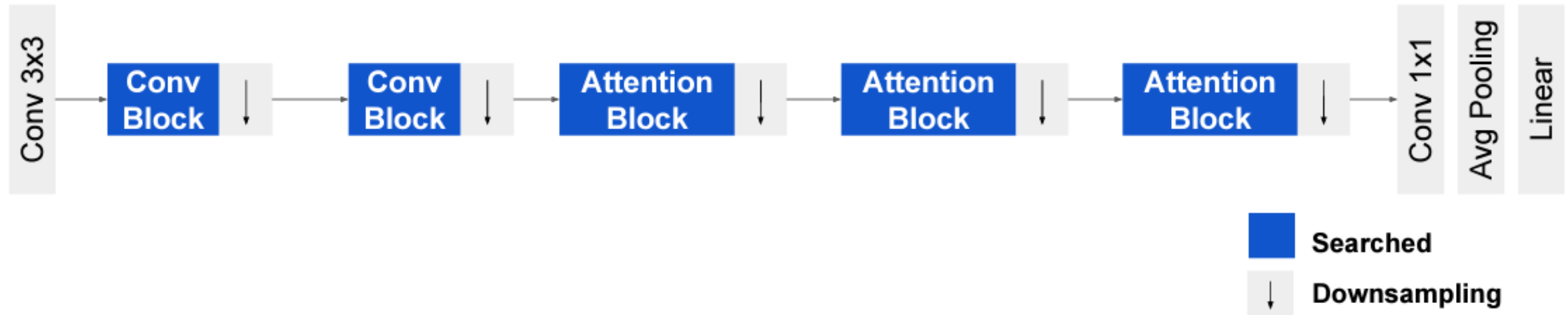


Institución
Universitaria
Reacreditada en Alta Calidad

squeezeNet

<https://vitalab.github.io/article/2018/03/15/squeezeNet.html>

HyT-NAS



Bibliografía

- H. Benmeziane et al. A Comprehensive Survey on Hardware-Aware Neural Architecture Search. 2021. <https://arxiv.org/pdf/2101.09336.pdf>
- <https://github.com/alibaba/lightweight-neural-architecture-search>



Institución
Universitaria
Reacreditada en Alta Calidad

¡Gracias!

Somos Innovación Tecnológica con *Sentido Humano*



Alcaldía de Medellín